Financial Copilot Mortgage & Liability Enhancement Blueprint

Step-by-step AI-agent instructions for modelling home loans, property assets, net worth, debt optimisation, and balance-sheet intelligence.

Purpose: This document instructs an AI coding agent how to enhance the Financial Copilot application so that it correctly treats home loans, mortgages, and other debts as liabilities - not investments - while modelling the linked property as an asset.

Core principle: A home loan is a liability. The property is the asset. The difference between the two is home equity.

1. Business Requirement

Enhance the Financial Copilot app from an investment tracker into a complete personal balance-sheet intelligence platform. The app must understand assets, liabilities, linked properties, debt repayment, equity growth, net worth, and the trade-off between investing and paying down debt.

Required outcome

- Users can add a property as an asset.
- Users can add a mortgage or home loan as a liability.
- A mortgage can be linked to a property asset.
- The app calculates home equity automatically.
- The app calculates total assets, total liabilities, and net worth.
- The app simulates extra bond payments and interest savings.
- The AI assistant can reason over the user balance sheet safely and explain options.

2. Correct Financial Model

Concept	Correct Treatment
Property / home	Asset
Home loan / mortgage / bond	Liability
Home equity	Property value minus bond balance
Bond overpayment	Debt reduction / interest saving
Access bond available balance	Available liquidity, not income

```
Net Worth = Total Assets - Total Liabilities

Home Equity = Property Current Value - Outstanding Home Loan Balance

Daily Bond Interest = Outstanding Balance * Annual Interest Rate / 365
```

3. Updated Domain Model

Do not force every financial item into an Investment table. The system must have first-class concepts for Asset and Liability. Investments are only one category of asset.

Core entities

```
User
Account
Asset
Liability
Holding
Transaction
CashFlow
Goal
Recommendation
MonthlySnapshot
ValuationHistory
InterestRateHistory
```

Asset types

```
ETF
EQUITY
CASH
PROPERTY
RETIREMENT
CRYPTO
COMMODITY
BUSINESS
VEHICLE
OTHER
```

Liability types

```
HOME_LOAN
VEHICLE_FINANCE
PERSONAL_LOAN
CREDIT_CARD
OVERDRAFT
STUDENT_LOAN
BUSINESS_LOAN
OTHER
```

4. Database Enhancement Instructions

Implement database migrations. Use clear constraints and foreign keys. The Spring Boot API remains the system of record. The AI orchestrator must call the API rather than writing directly to the database.

Suggested tables

```
assets
- id
- user_id
- account_id nullable
- asset_type
- name
- institution
- currency
- current_value
- valuation_method
- created_at
- updated_at
```

```
liabilities
- id
- user_id
- account_id nullable
- liability_type
- name
- institution
- currency
- outstanding_balance
- interest_rate
- minimum_payment
- payment_frequency
- original_term_months
- remaining_term_months
- start_date
- linked_asset_id nullable
- access_facility_enabled boolean
- available_redraw_amount nullable
- created_at
- updated_at
```

5. Backend API Requirements

Create versioned REST APIs. Keep them clean, testable, and documented with OpenAPI.

Asset endpoints

```
POST   /api/v1/assets
GET    /api/v1/assets
GET    /api/v1/assets/{id}
PUT    /api/v1/assets/{id}
DELETE /api/v1/assets/{id}
GET    /api/v1/assets/{id}/valuation-history
```

Liability endpoints

```
POST   /api/v1/liabilities
GET    /api/v1/liabilities
GET    /api/v1/liabilities/{id}
PUT    /api/v1/liabilities/{id}
DELETE /api/v1/liabilities/{id}
POST   /api/v1/liabilities/{id}/link-asset/{assetId}
GET    /api/v1/liabilities/{id}/amortisation
POST   /api/v1/liabilities/{id}/simulate-extra-payment
```

Balance sheet endpoints

```
GET /api/v1/balance-sheet/summary
GET /api/v1/balance-sheet/net-worth
GET /api/v1/balance-sheet/asset-allocation
GET /api/v1/balance-sheet/debt-ratio
GET /api/v1/balance-sheet/home-equity
```

6. Calculation Engine Requirements

All financial calculations must be deterministic and unit tested. The LLM must not calculate balances or interest directly. It may explain results returned by the calculation engine.

- Calculate daily interest using balance, rate, and day count.
- Calculate monthly interest estimates.
- Calculate amortisation schedule.

- Calculate payoff date at current payment.
- Calculate payoff date with extra payment.
- Calculate total interest saved.
- Calculate home equity.
- Calculate debt-to-asset ratio.
- Calculate net worth.
- Compare access bond benefit vs savings account yield.
- Compare bond overpayment vs ETF investing as a scenario, not as guaranteed advice.

7. Step-by-Step Enhancement Plan

Step	Enhancement	Deliverable
1	Add Asset and Liability models	Entities, enums, migrations
2	Add property asset support	Create/edit/list property assets
3	Add home loan liability support	Create/edit/list home loans
4	Link mortgage to property	Home equity calculation
5	Build net worth engine	Assets - liabilities summary
6	Build bond calculator	Interest, amortisation, payoff date
7	Add extra-payment simulator	R5k/R7k/R10k scenarios
8	Add access-bond logic	Available redraw and liquidity view
9	Enhance UI dashboard	Balance sheet and debt widgets
10	Expose AI tools	AI can call calculations via API
11	Add AI explanations	Plain-language insights
12	Add tests and reports	Unit, integration, sample data

8. Frontend UI Enhancements

Add new screens and widgets to the React/Next.js frontend. Keep the design clean, premium, and dashboard-oriented.

- Balance Sheet page showing assets, liabilities, and net worth.
- Property card showing property value, linked bond, and equity.
- Debt dashboard showing all liabilities and repayment progress.
- Bond optimiser screen with extra payment sliders.
- Access bond liquidity widget showing available redraw balance.
- Net worth chart over time.
- Debt-to-asset ratio chart.
- Scenario comparison cards for savings account vs access bond.

9. AI Orchestrator Instructions

The Python AI orchestrator must not directly own financial calculations. It must call Spring Boot tools/endpoints and narrate the results.

```
Available AI tools:
- get_balance_sheet_summary(userId)
- get_home_equity(userId, propertyId)
- simulate_bond_extra_payment(userId, liabilityId, extraAmount)
- compare_cash_in_bond_vs_savings(userId, amount, savingsRate)
- get_debt_to_asset_ratio(userId)
- generate_monthly_financial_insight(userId)

AI rule:
Never invent balances, interest savings, or payoff dates.
Always retrieve deterministic results from the API.
```

10. AI Assistant Behaviour

The assistant must explain mortgage and asset-liability concepts in plain language. It must avoid presenting regulated financial advice as certainty.

- Explain that property is an asset and home loan is a liability.
- Explain home equity clearly.
- Explain daily interest on bonds.
- Explain how access bond deposits reduce interest.
- Present scenarios, not commands.
- Mention assumptions clearly.
- Avoid guaranteed investment return claims.
- Recommend consulting a licensed financial advisor for personalised decisions.

11. Sample User Scenario for Testing

Field	Value
Property value	R2,500,000
Bond balance	R1,737,341.55
Bond interest rate	9.4%
Remaining term	210 months
Current instalment	R17,622/month
Extra payment scenario	R5,000 or R7,000/month
Emergency cash	R120,000
Savings rate comparison	6.6%

12. Acceptance Criteria

- A mortgage is stored as a liability, never as an asset.
- A property is stored as an asset.

- A liability can optionally link to an asset.
- Net worth calculation uses assets minus liabilities.
- Home equity calculation is correct and tested.
- Bond amortisation and extra-payment simulations are deterministic and unit tested.
- Frontend displays assets and liabilities separately.
- AI assistant calls backend tools instead of inventing numbers.
- OpenAPI documentation is updated.
- All migrations, services, controllers, DTOs, and tests are included.

13. Final Instruction to the AI Coding Agent

Implement this enhancement incrementally. Do not rewrite the whole platform. Create small pull requests or commits per step. Keep the Spring Boot API as the system of record. Keep the AI orchestrator as the explanation and reasoning layer. Prioritise correctness, testability, and clean domain modelling over speed.

```
Build order:
1. Database migrations
2. Backend entities and repositories
3. Services and calculation engine
4. REST controllers and OpenAPI docs
5. Unit tests
6. Frontend pages and widgets
7. AI orchestrator tools
8. AI assistant prompts
9. Integration tests
10. Final sample-data demo
```

End of document. This blueprint is ready to be uploaded to an AI coding agent such as Cursor, Devin, or ChatGPT Agent.